



ТЕСТИРОВАНИЕ, ОТЛАДКА И ИНТЕГРАЦИЯ ФУНКЦИОНАЛЬНЫХ КОМПОНЕНТОВ



История Тестирования

- Тестирование зародилось практически одновременно с программированием и долгое время считалось, что его целью является доказательство отсутствия ошибок в программе. Однако этот тезис не выдерживал критики, ведь для перебора всех возможных вариантов выполнения даже самой небольшой программы требуются титанические ресурсы.
- 

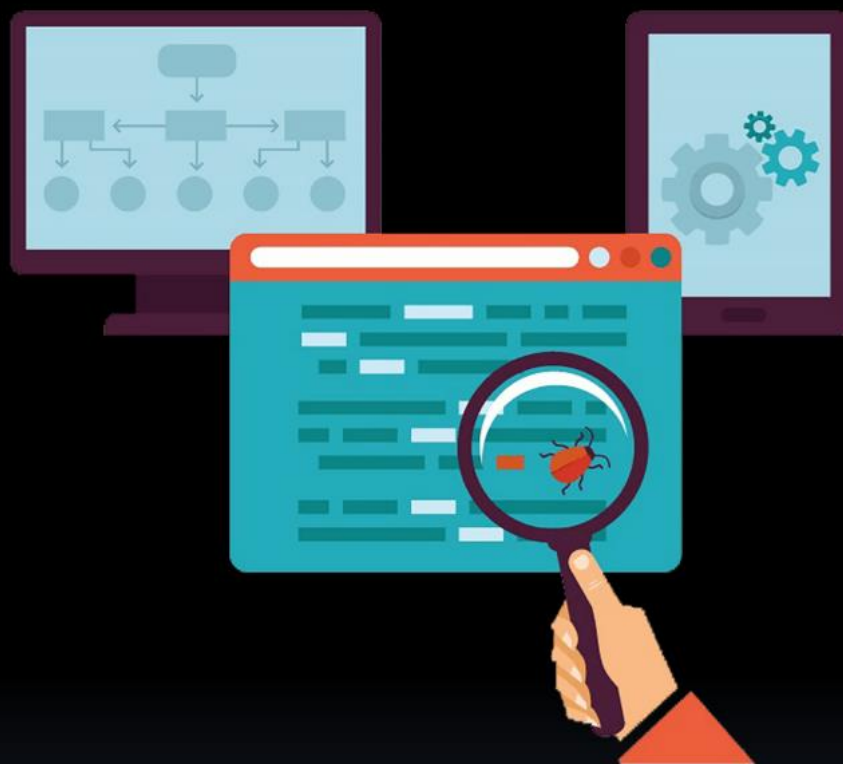
Тестирование: общие сведения



«Тестирование – это процесс выполнения программ с целью обнаружения ошибок»

- Гленфорд Майерс

Тестирование: общие сведения



- Классическое определение Майерса, приведенное выше, слишком узкое и не охватывает всех аспектов современного тестирования. Корректнее будет охарактеризовать тестирование ПО как процесс (т.е. плановая и упорядоченная деятельность) анализа или эксплуатации программного обеспечения с целью выявления ошибок, недочетов и несоответствий спецификациям до того, как продукт будет выпущен на рынок.

Классификации тестирования

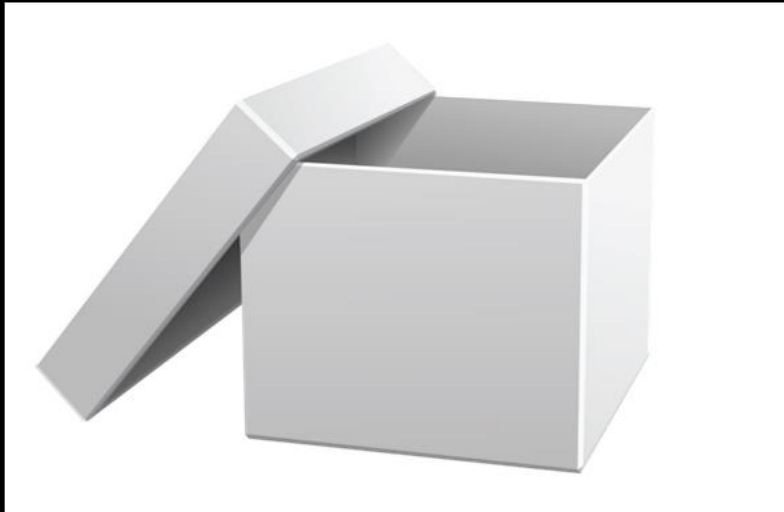


Динамическое тестирование (Dynamic Testing) - тестовая деятельность, предусматривающая эксплуатацию программного продукта

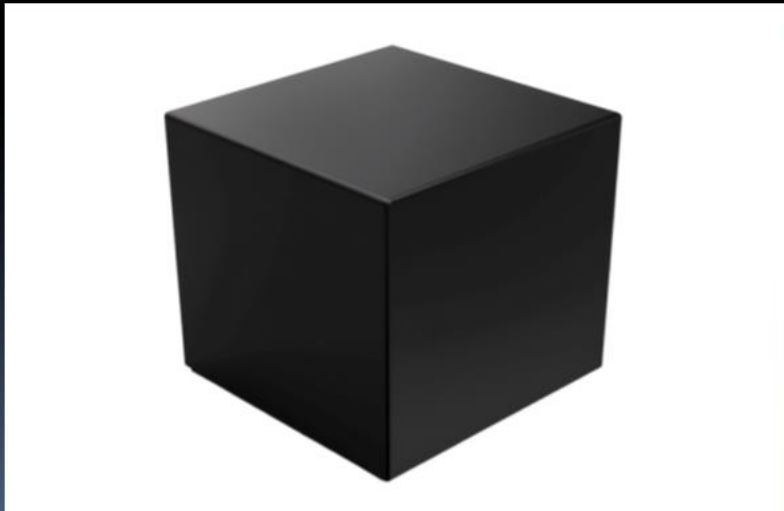


Статическое Тестирование (Static Testing) - тестовая деятельность, связанная с анализом программного обеспечения (ПО) и проводимая без его запуска (К статическому тестированию относятся обзоры кода, инспекции, аудит, критический анализ и т. д.)

Классификации тестирования



Метод белого ящика - изучение внутреннего устройства программы (исходных текстов, модулей и их структур).



Метод чёрного ящика - программе подаются некоторые данные на вход и проверяются результаты в надежде найти несоответствия. При этом, как именно работает программа, считается несущественным. Этот подход до сих пор является самым распространенным в повседневной практике.

Классификации тестирования



Метод серого ящика - имеется частичный доступ к исходному коду (например, сторонние компоненты или автоматизация функционального тестирования через пользовательский интерфейс).

Классификации тестирования



Ручное тестирование - тест-кейсы выполняет человек

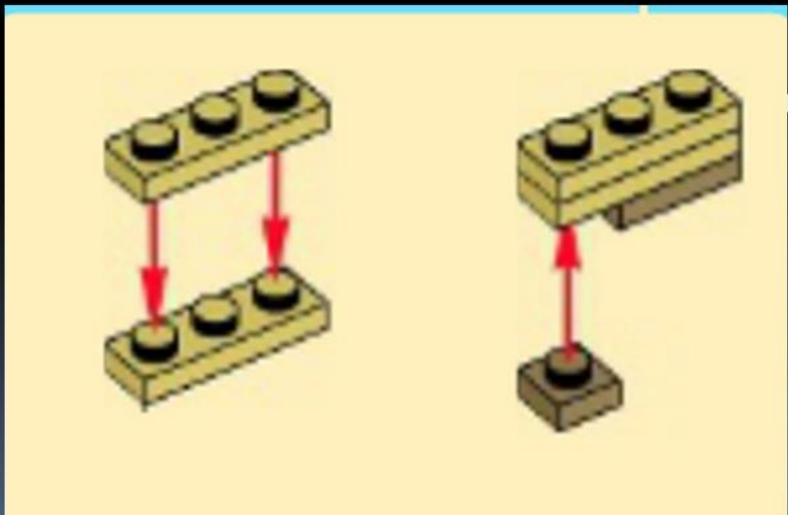


Автоматизированное тестирование - тест-кейсы частично или полностью выполняет специальное инструментальное средство.

Классификации тестирования



Модульное тестирование - проверяются отдельные небольшие части приложения



Интеграционное тестирование - проверяется взаимодействие между несколькими частями приложения

Классификации тестирования



Системное тестирование -
приложение проверяется как единое
целое

Классификации тестирования



Позитивное тестирование - все действия с приложением выполняются строго по инструкции без никаких недопустимых действий, некорректных данных и т. д. Можно образно сказать, что приложение исследуется в «тепличных условиях»



Негативное тестирование - в работе с приложением выполняются (некорректные) операции и используются данные, потенциально приводящие к ошибкам

Отладка



- Отладка – это процесс обнаружения причин возникновения ошибок и их последующего исправления



Отладка

- Методику отладки можно описать в виде последовательности следующих этапов:

- 1) Стабилизация ошибки (1-й шаг обобщенного научного подхода).
 - 2) Обнаружение точного места ошибки (2–4 шаги обобщенного научного подхода).
 - 3) Исправление ошибки.
 - 4) Проверка исправления.
 - 5) Поиск похожих ошибок.
- 

Инструменты отладки

1) Отладчики (Debuggers)

Позволяют разработчикам исполнять программу пошагово, останавливая её выполнение на определенных точках (брейкпоинты), чтобы изучить текущее состояние программы. Это включает в себя возможность просмотра значений переменных, состояние стека вызовов и поток выполнения программы



Инструменты отладки

2) Логирование (Logging)

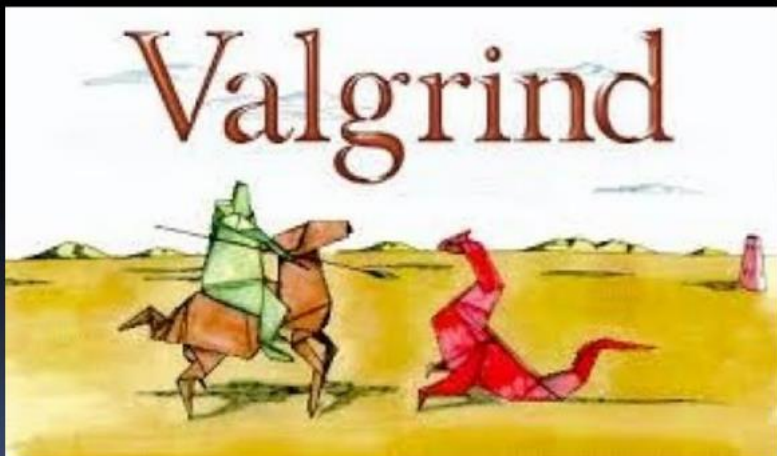
Процесс записи событий, происходящих в программе, во внешние файлы или системы хранения логов. Логи могут содержать информацию о работе программы, предупреждениях и ошибках. Логирование полезно для отслеживания поведения программы в реальном времени и для постфактум-анализа проблем.



Инструменты отладки

3) Диагностические инструменты

Диагностические инструменты помогают анализировать программу для выявления причин проблем, не всегда непосредственно связанных с ошибками в коде, например, утечки памяти или проблемы с производительностью



DevTools

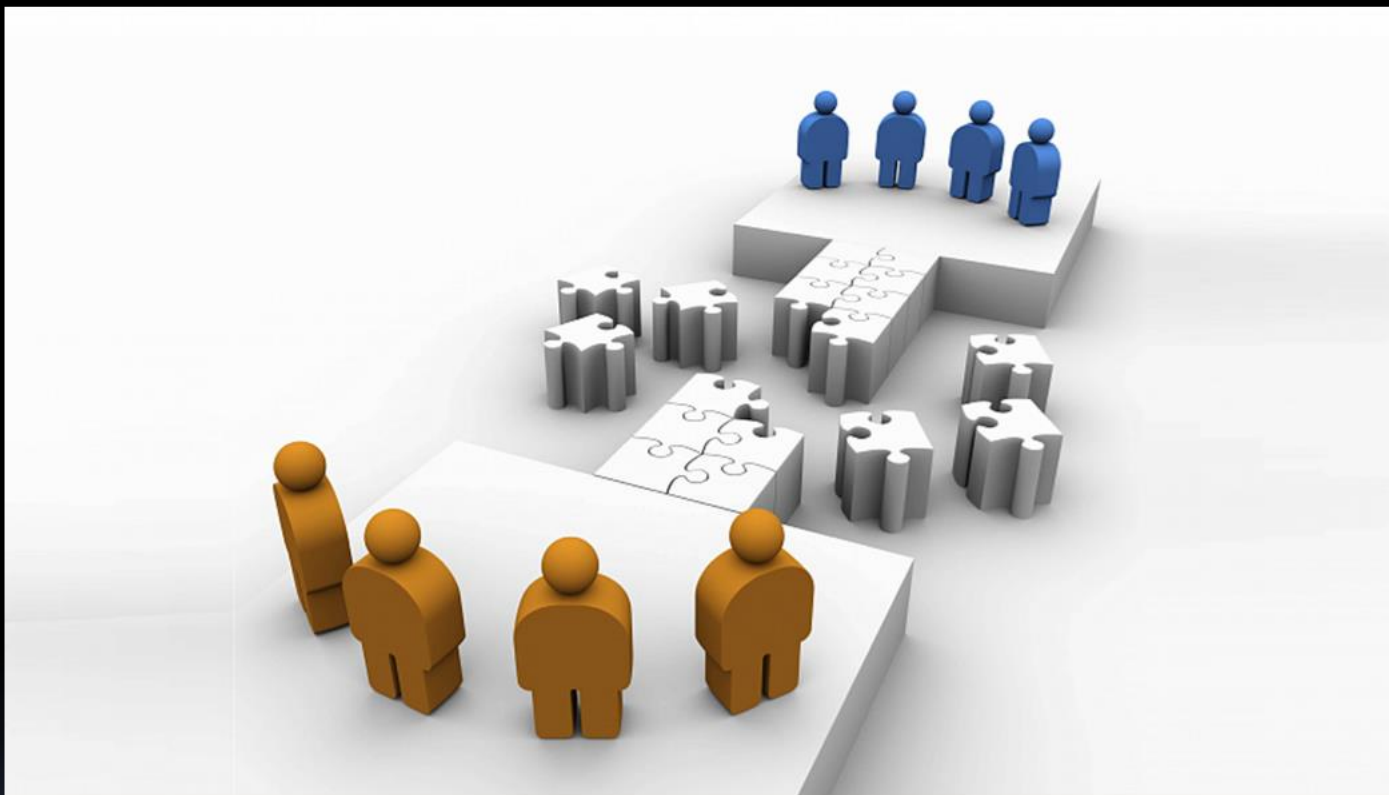


Интеграция

Интеграция функциональных компонентов — это процесс объединения отдельных частей системы (модулей, сервисов, программных блоков) в единую работающую систему, где все компоненты взаимодействуют корректно, обмениваются данными и выполняют общие функции.

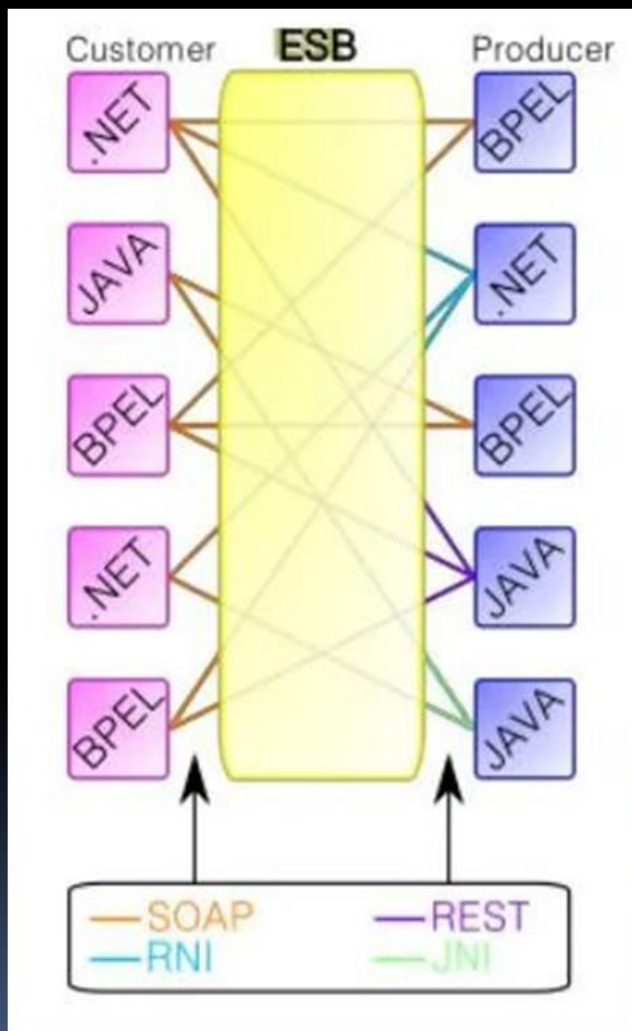


Способы интеграции компонентов



1) Горизонтальная интеграция - компоненты одного уровня взаимодействуют напрямую.

Способы интеграции компонентов



3) Интеграция через шину (ESB — Enterprise Service Bus)- все компоненты подключаются через единую коммуникационную шину. Используется в больших корпоративных системах.

Способы интеграции компонентов

4) Интеграция через API (Application Programming Interface) - стандартный современный подход



{ REST-API }



Проблемы, которые решает интеграция:

- Несоответствие форматов данных,
 - Несовпадение интерфейсов и протоколов,
 - Ошибки синхронизации.
- 

ИСТОЧНИКИ

https://rep.bntu.by/bitstream/handle/data/70333/Otladka_i_testirovanie_programmnogo_obespecheniya.pdf?sequence=1

https://rep.bntu.by/bitstream/handle/data/73188/Testirovani_e_i_otladka_PO.pdf?sequence=1&isAllowed=y

<https://cyberleninka.ru/article/n/sistemnoe-testirovanie-programmnyh-izdeliy>